Universidad San Carlos de Guatemala – CUNOC

División de Ciencias de la Ingeniería

Organización de Lenguajes y Compiladores 2

Ingeniero Mario Tobar

Auxiliar Yennifer de León

Sección “A”



Manual Tecnico

Juan Pablo de León Miranda

202031101

Quetzaltenango, octubre de 2026

Índice

Tecnologías Usadas....3
Requerimientos del Sistema.... 3
Diagrama de Clases....3
Semántico.... 3
Generación Código.... 4
Especificaciones de Lenguajes....4
Símbolos Común....4
Agrupación.... 4
Puntuación.... 4
Relacionales....5
Lógicos....5
Aritméticos y Asignación.... 5
Comentario Simple....6
PigLatin.... 6
Palabras Reservadas....6
Tipos de Datos.... 6
Gramática....7
Zetariano.... 10
Palabras Reservadas.... 10
Tipos de Datos.... 10
Símbolos.... 10
Gramática.... 11
Y?.... 14
Palabras Reservadas.... 14
Tipos de Datos.... 14
Símbolos.... 15
Gramática....16
Compatibilidad de tipos.... 20
Equivalencia de Tipos.... 20
Asignación de Aceptación....20
Resultados en Operaciones....21

Tecnologías Usadas

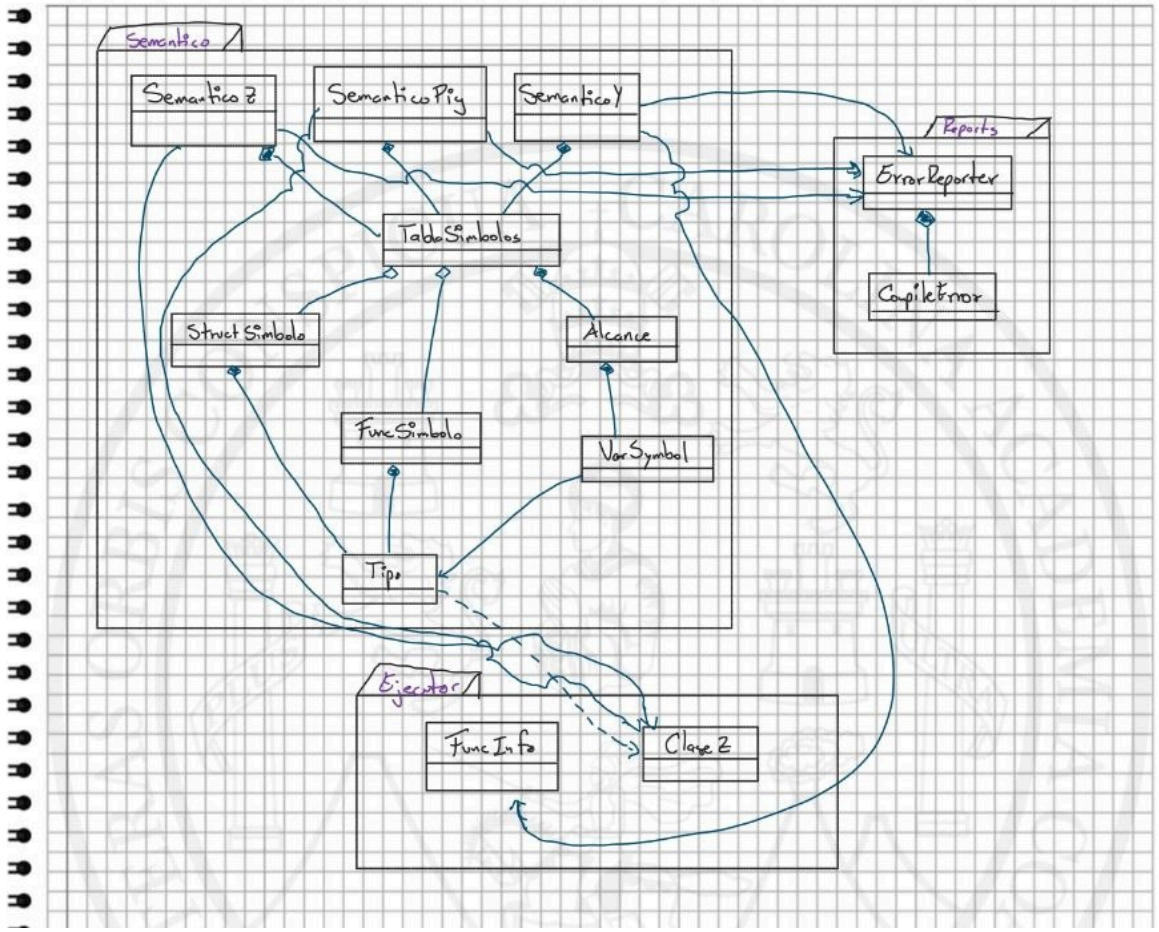
- Ide: IntelliJ IDEA 2026.1.3
- Antlr4 4.13.2: Enlace descarga maven version
- rars 1.6: Enlace descarga jar

Requerimientos del Sistema

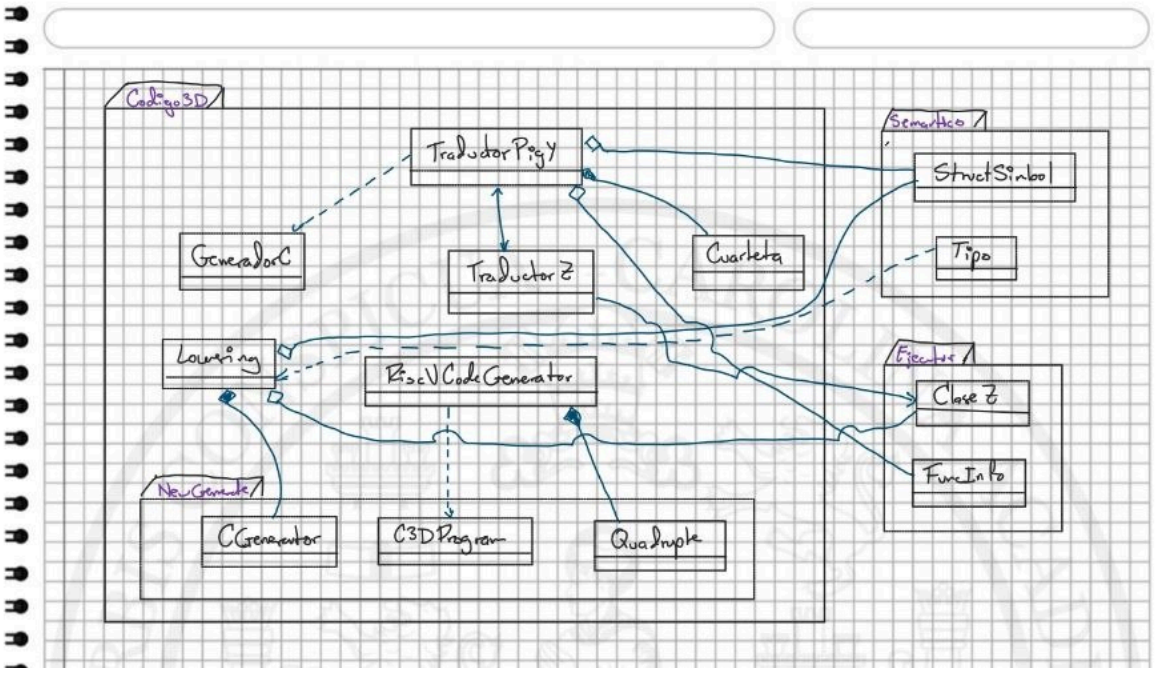
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment (build 21.0.12.1+1-1-24.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-24.04.4-Ubuntu, mixed mode, sharing)

Diagrama de Clases

Semántico



Generación Código



Especificaciones de Lenguajes

Símbolos Común

Agrupación

Símbolo	Nombre
(	LPAREN
)	RPAREN
[	LCORCH
]	RCORCH
{	LLLAVE
}	RLLAVE

Puntuación

Símbolo	Nombre
:	DOS_PUNTOS
,	COMA

.	PUNTO
;	PUNTO_COMA

Relacionales

Símbolo	Nombre
==	IGUAL
!=	NO_IGUAL
<=	MENOR_IGUAL
>=	MAYOR_IGUAL
<	MENOR
>	MAYOR

Lógicos

Símbolo	Nombre
&&	AND
||	OR

Aritméticos y Asignación

Símbolo	Nombre
+	SUMA
-	RESTA
/	DIV
*	MULTI
=	ASIGNACION
++	SUMA_INCR
–	RESTA_DECR

Comentario Simple

Símbolo	Nombre
//	LINE_COMMENT

PigLatin

Palabras Reservadas

- import
- VARIABILES>
- esto
- bool
- numerus
- textum
- MAIOR>
- FINIS
- decimalis
- littera
- series
- falsus
- verum
- null
- novus
- si
- finis
- aliter
- dum
- facere
- per
- non
- perge
- interrumpe
- >>
- <<

Tipos de Datos

- Littera: Carácter
- Numerus: números enteros
- Decimalis: números decimales
- Bool: boleano
- Textum: Cadena de Texto

Tipo	**Valor**
ID	[a-zA-Z_][a-zA-Z_0-9]*
INT	[0-9]+
DOUBLE	INT '.' INT
STRING	"" (ESC|.)*? ""
CHARS	'\" [a-z-A-Z] '\"
ESC	'\\"' | '\\\\'
BLOCK_COMMENT	'##' .*? '##'

Gramática

```
inicio: instrucciones? EOF
      ;

instrucciones: bloque_imports* vars_par? bloque_main FINISUP PUNTO_COMA
             ;

bloque_imports: IMPORT ID (PUNTO ID)*
              ;

vars_par: VARIABILES bloque_vars*
        ;

        // esto <id> : <tipo> <valor> ;
bloque_vars: ESTO ID DOS_PUNTOS tipos expresion PUNTO_COMA
          // esto <id> : novus <id_objeto>(...) ;
           | ESTO ID DOS_PUNTOS NOVUS ID LPAREN bloque_objt? RPAREN PUNTO_COMA
          // esto <id> : novus <id_objeto>(...) : <tipo_estatico> ;
           | ESTO ID DOS_PUNTOS NOVUS ID LPAREN bloque_objt? RPAREN DOS_PUNTOS tipos_varios PUNTO_COMA
          // esto <id> : null : <tipo_estatico> ;
           | ESTO ID DOS_PUNTOS NULL_VAL DOS_PUNTOS tipos_varios PUNTO_COMA
          // esto <id> : <expresion> : <tipo_estatico> ;
           | ESTO ID DOS_PUNTOS expresion DOS_PUNTOS tipos_varios PUNTO_COMA
          // esto <id> : <valor> ; <- Solo para expresiones booleanas
           | ESTO ID DOS_PUNTOS expresion PUNTO_COMA
           // -------------- ARRAYS ----------------------
          // series <id>[<valor>] : <tipo> {...} ;
           | SERIES ID LCORCH expresion RCORCH DOS_PUNTOS tipos_varios LLLAVE bloque_varios RLLAVE PUNTO_COMA
          // series <id>[<valor>] : <tipo> ;
           | SERIES ID LCORCH expresion RCORCH DOS_PUNTOS tipos_varios PUNTO_COMA
           // -------------- ESTRUCTURAS --------------------
          // esto <id> : <id_estructura> {...} ;
           | ESTO ID DOS_PUNTOS ID LLLAVE bloque_varios RLLAVE PUNTO_COMA
           // ---------------- OBJETOS ----------------------
          // series <id>.[<valor>] : <id_objeto> ;
           | SERIES ID LCORCH expresion RCORCH DOS_PUNTOS ID PUNTO_COMA
           ;
```

```
tipos_varios: tipos
           | ID
           ;

bloque_varios: (expresion | LLLAVE bloque_varios RLLAVE)  (COMA (expresion | LLLAVE bloque_varios RLLAVE))*
             ;

bloque_objt: (expresion | NOVUS ID LPAREN bloque_objt? RPAREN) (COMA (expresion | NOVUS ID LPAREN bloque_objt? RPAREN))*
           ;


bloque_main: MAIOR instruccion*
           ;

instruccion: bloque_impr
           | bloque_leer
           | expresion PUNTO_COMA?
           | SI LPAREN expresion RPAREN LLLAVE instruccion* RLLAVE bloque_si FINIS PUNTO_COMA
           | DUM LPAREN expresion RPAREN LLLAVE instruccion+ RLLAVE FINIS PUNTO_COMA
           | FACERE LLLAVE instruccion+ RLLAVE DUM LPAREN expresion RPAREN PUNTO_COMA
           | PER LPAREN ESTO ID DOS_PUNTOS tipos expresion PUNTO_COMA expresion PUNTO_COMA auto_cambio RPAREN LLLAVE instruccion+ RLLAVE
           | bloque_asignacion
           | PERGE PUNTO_COMA
           | INTERRUMPE PUNTO_COMA
           ;
```

```
bloque_si: (ALITER LPAREN expresion RPAREN LLLAVE instruccion* RLLAVE)* (ALITER LLLAVE instruccion* RLLAVE)?
         ;

        // >> <valor> ; | >> <valor>  ...  >> <valor> ;
bloque_impr: IMPRIMIR expresion (IMPRIMIR expresion)* PUNTO_COMA # Impresion_Consola
        ;

      // <id> << | <<
bloque_leer: ID? LEER # Lectura_Texto
        ;

               // <id> = <valor> ;
bloque_asignacion: ID ASIGNACION expresion PUNTO_COMA
               // <id>[<index>] = <valor> ;
                | ID LCORCH expresion RCORCH ASIGNACION expresion PUNTO_COMA
               // <id>.<id> = <valor> ;
                | ID PUNTO ID ASIGNACION expresion PUNTO_COMA
               // <id>[<index>].<id> = <valor> ;
                | ID LCORCH expresion RCORCH PUNTO ID ASIGNACION expresion PUNTO_COMA
                | auto_cambio PUNTO_COMA
                ;

          // <id> ++
auto_cambio: ID SUMA_INCR
         // <id> --
          | ID RESTA_DECR
          ;
```

```
        // - <valor>
expresion: RESTA expresion # Umenos
        // non <valor>
         | NON expresion # Negacion
        // (...)
         | LPAREN expresion RPAREN # Parentesis
        // <exp>.<id>(...)
         | expresion PUNTO ID LPAREN (expresion (COMA expresion)* )? RPAREN # Metodo_Cadena
        // <exp>.<id>
         | expresion PUNTO ID # Acceso_Cadena
        // <exp>[<valor>]
         | expresion LCORCH expresion RCORCH # Acceso_Indice
        // <valor> (* /) <valor>
         | expresion ops1=(MULTI | DIV) expresion # MultDiv
        // <valor> (+ -) <valor>
         | expresion ops1=(SUMA | RESTA) expresion # SumaResta
        // <valor> (== !=) <valor>
         | expresion ops1=(IGUAL | NO_IGUAL) expresion # IgualNoIgual
        // <valor> (<= >=) <valor>
         | expresion ops1=(MENRO_IGUAL | MAYOR_IGUAL) expresion # MenorMayorIgual
        // <valor> (< >) <valor>
         | expresion ops1=(MENOR | MAYOR) expresion # MenorMayor
        // <valor> (&& ||) <valor>
         | expresion ops1=(AND | OR) expresion # AndOr
        // <id> (...)
         | ID LPAREN (expresion (COMA expresion)* )? RPAREN # Llamada_Actio_Exp
        // <tipo> <id> (...)
         | tipos ID LPAREN (expresion (COMA expresion)* )? RPAREN # Llamada_Ratio_Tipo
         | NOVUS ID LPAREN (expresion (COMA expresion)* )? RPAREN # Novus_Exp
         | VERUM # VerumValor
         | FALSUS # FalsusValor
         | NULL_VAL # NullVal
         | ID # Identificador
         | DOUBLE # DoubleVal
         | INT # IntVal
         | CHARS # CharVal
         | STRING # StringVal
         ;
```

```
tipos: NUMERUS
     | TEXTUM
     | BOOL
     | DECIMALIS
     | LITTERA
     ;
```

Zetariano

Palabras Reservadas

- public
- class
- String
- int
- boolean
- double
- char
- void
- return
- new
- if
- else
- true
- false
- switch
- case
- default
- break
- for
- continue
- while
- do
- println
- print
- readln
- null
- private
- protected
- @Override
- this
- super
- extends

Tipos de Datos

- String: Cadena de Texto
- Int: Numeros enteros
- Boolean: Booleano
- Double: Números decimales
- Char: Caracter

Tipo	Valor
ID	[a-zA-Z_][a-zA-Z_0-9]*
INTEGER	[0-9]+
DOUBLES	INTEGER '.' INTEGER
STRINGS	"" (ESC|.)*? ""
CHARS	'\" [a-z-A-Z] '\"
ESC	'\\"' | '\\\\'
BLOC_COMMENT	'/*' .*? '*/'

Símbolos

Símbolo	Nombre
?	INTERRG
!	NOT

+=	SUMA_IGL
-=	RESTA_IGL
*=	MULT_IGL
%	MOD

Gramática

```
inicio: bloc_main EOF
      ;

       // public class <id> {...}
bloc_main: PUBLIC CLASS ID (EXTENDS ID)? LLLAVE instrucciones* RLLAVE
            ;

          // public <id> (...){...} <- Puede o no tener parametros, y puede o no tener instrucciones dentro
instrucciones: OVERRIDE? modificador? ID LPAREN param_var? RPAREN LLLAVE instruccion* RLLAVE
           // [modificador] void <id> (...) {...}
            | OVERRIDE? modificador? VOID ID LPAREN param_var? RPAREN LLLAVE instruccion* RLLAVE
           // [modificador] <tipo> <id> (...) {...}
            | OVERRIDE? modificador? (tipos | ID) ID LPAREN param_var? RPAREN LLLAVE instruccion* RLLAVE
           //declaracion de variables
            | modificador? declaracion+
            ;

modificador: PUBLIC
           | PRIVATE
           | PROTECTED
           ;

encapsulamiento: modificador
               ;
```

```
instruccion: declaracion
         | asignacion
       // if (<exp bool> ) {...} ...
         |IF LPAREN expresion RPAREN LLLAVE instruccion* RLLAVE bloque_si
       // if (<exp bool>) ...
         | IF LPAREN expresion RPAREN instruccion
         // if (<exp bool>) .... else ....
         | IF LPAREN expresion RPAREN instruccion ELSE instruccion
       // switch (<opcion>) { ... } || switch (<opcion>) case ... default ...
         | SWITCH LPAREN expresion RPAREN switch_cuerpo
         // for (<tipo> <id> = <valor> ; <exp bool> ; <id> ++ o --) {....} || for ( ; ; ) {....}
         | FOR LPAREN ((tipos ID ASIGNACION expresion PUNTO_COMA expresion PUNTO_COMA ID (INCREMENTO | DECREMENTO)) | (PUNTO_COMA PUNTO_COMA)) RPAREN (LLLAVE instruccion+ RLL
         // while (<exp bool>) {...}
         | WHILE LPAREN expresion RPAREN (LLLAVE instruccion+ RLLAVE | instruccion)
         // do {...} while (<exp bool>) ;
         | DO LLLAVE instruccion+ RLLAVE WHILE LPAREN expresion RPAREN PUNTO_COMA
       // println(...);
         | PRINTLN LPAREN expresion? RPAREN PUNTO_COMA
       // print(...);
         | PRINT LPAREN expresion? RPAREN PUNTO_COMA
       // readln() ;
         | READLN LPAREN RPAREN PUNTO_COMA
       // break ;
         | BREAK PUNTO_COMA
       // continue ;
         | CONTINUE PUNTO_COMA
       // return <valor> ; || return ;
         | RETURN expresion? PUNTO_COMA
       // super(...) ; <- debe ser la primera instruccion del constructor
         | SUPER LPAREN params? RPAREN PUNTO_COMA
         | expresion PUNTO_COMA
         ;
```

```
        // else if (<exp bool> ) {...} || else {...}
bloque_si: (ELSE IF LPAREN expresion RPAREN LLLAVE instruccion* RLLAVE)* (ELSE LLLAVE instruccion* RLLAVE)?
         ;

switch_cuerpo: LLAVE bloque_switch* bloque_default? RLLAVE
             | bloque_switch+ bloque_default?
             ;

           // case <id> : ... break; <- Puede no tener break
bloque_switch: CASE expresion DOS_PUNTOS instruccion*
             ;

bloque_default: DEFAULT DOS_PUNTOS instruccion*
              ;

          // <tipos> <id> = <value> ;
declaracion: (tipos | ID) ID ASIGNACION expresion PUNTO_COMA
          // <tipos>[] <id> = new <tipos>[<value>] ; || <tipos>[][]... <id> = new <tipos>[<value>][<value>]... ; <- misma cantidad en ambos lados
          | (tipos | ID) (LCORCH RCORCH)+ ID ASIGNACION NEW tipos (LCORCH expresion RCORCH)+ PUNTO_COMA
         // <tipos> [] <id> = {...} ;
          | (tipos | ID) LCORCH RCORCH ID ASIGNACION LLLAVE params RLLAVE PUNTO_COMA
         // <tipos>[]... <id> ;
          | (tipos | ID) (LCORCH RCORCH)+ ID PUNTO_COMA
         // <tipos> <id> ;
          | (tipos | ID) ID PUNTO_COMA
         // <tipos> <id> = new
          | (tipos | ID) ID ASIGNACION NEW ID LPAREN params? RPAREN PUNTO_COMA
          ;
```

```
         // <id> += <value> ;
asignacion: THIS PUNTO ID SUMA_IGL expresion PUNTO_COMA
          | THIS PUNTO ID RESTA_IGL expresion PUNTO_COMA
          | THIS PUNTO ID MULT_IGL expresion PUNTO_COMA
          | THIS PUNTO ID ASIGNACION expresion PUNTO_COMA
          | THIS PUNTO ID INCREMENTO PUNTO_COMA
          | THIS PUNTO ID DECREMENTO PUNTO_COMA
        // <id> += <value> ;
          | ID SUMA_IGL expresion PUNTO_COMA
        // <id> -= <value> ;
          | ID RESTA_IGL expresion PUNTO_COMA
        // <id> *= <value> ;
          | ID MULT_IGL expresion PUNTO_COMA
        // <id> = (<exp bool>) ? <exp> : <exp> ;
          | ID ASIGNACION LPAREN expresion RPAREN INTERRG expresion DOS_PUNTOS expresion PUNTO_COMA
        // <id> = <valor> ;
          | lvalue ASIGNACION expresion PUNTO_COMA
        // <id> ++ ;
          | ID INCREMENTO PUNTO_COMA
        // <id> -- ;
          | ID DECREMENTO PUNTO_COMA
          ;

lvalue: ID (PUNTO ID | LCORCH expresion RCORCH)*
      ;
      // <id> || <id>, <id>, ... , <id>
params: expresion (COMA expresion)*
      ;

        // <tipo> <id> || <tipo> <id>, <tipo> <id>, ... , <tipo> <id>
param_var: (tipos | ID) expresion (COMA (tipos | ID) expresion)*
         ;
```

```
expresion: // <exp>.<id>
           expresion PUNTO ID
         // <exp>.<id>(...)
         | expresion PUNTO ID LPAREN params? RPAREN
         // <exp>[<valor>]
         | expresion LCORCH expresion RCORCH
        // - <valor>
         | RESTA expresion
        // ! <exp>
         | NOT expresion
        // (...)
         | LPAREN expresion RPAREN
         // <cond> ? <valor> : <valor>
         | expresion INTERRG expresion DOS_PUNTOS expresion
         // new <tipo>[<valor>]...
         | NEW tipos (LCORCH expresion RCORCH)+
         // new <Clase>[<valor>]...  (arreglo de objetos)
         | NEW ID (LCORCH expresion RCORCH)+
         // new <id>(...)
         | NEW ID LPAREN params? RPAREN
       // <valor> * / <valor>
         | expresion (MULT | DIV) expresion
       // <valor> % <valor>
         | expresion MOD expresion
       // <valor> + - <valor>
         | expresion (SUMA | RESTA) expresion
         // <valor> (== !=) <valor>
         | expresion ops1=(IGUAL | NO_IGUAL) expresion //# IgualNoIgual
         // <valor> (<= >=) <valor>
         | expresion ops1=(MENOR_IGUAL | MAYOR_IGUAL) expresion //# MenorMayorIgual
         // <valor> (< >) <valor>
         | expresion ops1=(MENOR | MAYOR) expresion //# MenorMayor
         // <valor> (&& ||) <valor>
         | expresion ops1=(AND | OR) expresion //# AndOr
```

```
        // this.<expresion>
         | THIS PUNTO expresion
        // super.metodo(...) / super.atributo
         | SUPER PUNTO ID LPAREN params? RPAREN
         | SUPER PUNTO ID
         | THIS
        // <tipo> <id> (...)
         | ID LPAREN params? RPAREN
         | TRUE
         | FALSE
         | NULL_VAL
         | ID
         | DOUBLES
         | INTEGER
         | CHARS
         | STRINGS
         ;

tipos: INT
     | DOUBLE
     | BOOLEAN
     | CHAR
     | STRING
     ;
```

Y?

Palabras Reservadas

- %estructuras
- estructura
- cadena
- entero
- flotante
- caracter
- bool
- %funciones
- definir
- si
- entonces
- sino
- contrario
- elegir
- caso
- romper
- para
- continuar
- mientras
- hacer
- imprimir
- leer
- verdadero
- falso
- retornar
- siempre

Tipos de Datos

- Cadena: Cadena de Texto

- Entero: Numeros Enteros
- Flotante: Numeros Decimales
- Caracter: Caracter
- Bool: Booleano

Tipo	Valor
ID	[a-zA-Z_][a-zA-Z0-9_]*
INT	[0-9]+
DECIMAL	INT '.' INT
STRING	'"' (ESC | ~["\\\r\n])* '"'
CHAR	'\'' (ESC | ~['\\\r\n]) '\''
fragment ESC	'\\' [btnrf"'\\]
BLOCK_COMMENT	'/*' .*? '*/'
NEWLINE	'\r'? '\n' | '\r'
TAB	[\t]+

Símbolos

Símbolo	Nombre
->	RETORNO_FUNC
!	NOT
+=	ADD_ASSIG
-=	SUB_ASSIG
*+	MUL_ASSIG
/=	DIV_ASSIG
%	MOD

Gramática

```
inicio: NEWLINE* bloq_estruc? bloq_func NEWLINE* EOF
      ;
          // %estructuas ...
bloq_estruc: ESTRUCTURAS NEWLINE (INDENT esctruc+ DEDENT | esctruc*)
           ;

        // estructua <id> : ...
esctruc: ESTRUCTURA ID DOS_PUNTOS NEWLINE INDENT definiciones+ DEDENT
       ;

        // %funciones ...
bloq_func: FUNCIONES NEWLINE (INDENT bloc_func+ DEDENT | bloc_func*)
         ;

        // definir <id> (...) -> <tipo> : ....
bloc_func: DEFINIR ID LPAREN params? RPAREN RETORNO_FUNC (tipos | ID) DOS_PUNTOS NEWLINE bloque
        // definir <id> (...) :
         | DEFINIR ID LPAREN params? RPAREN DOS_PUNTOS NEWLINE bloque
         ;

params: tipos_params (COMA tipos_params)*
      ;

            // <tipo> <id>
tipos_params: (tipos | ID) ID
           // [] <tipo> <id>
           | LCORCH RCORCH (tipos | ID) ID
           // {} <tipo> <id>
           | LLLAVE RLLAVE (tipos | ID) ID
           ;
```

```
          // {...}, {...} .... , {...}
bloc_llaves: LLAVE val_arreglo RLLAVE (COMA LLLAVE val_arreglo RLLAVE)*
          ;

          // <val> , <val> , ... , <val>
val_arreglo: expresion (COMA expresion)*
          ;


instrucciones: definiciones
            | asignaciones
            // llamada suelta: funcion(...) o objeto.metodo(...)
            | llamada NEWLINE
            // leer() como instruccion
            | LEER LPAREN RPAREN NEWLINE
            // linea en blanco dentro de un bloque
            | NEWLINE
            // imprimir ()
            | IMPRIMIR LPAREN expresion? RPAREN NEWLINE
            // si(<exp bool>) entonces ...
            | SI LPAREN expresion RPAREN ENTONCES NEWLINE bloque bloc_si
            // mientras(<exp bool>) hacer ...
            | MIENTRAS LPAREN expresion RPAREN HACER NEWLINE bloque
            // hacer: ... mientras(<exp bool>)
            | HACER DOS_PUNTOS NEWLINE bloque MIENTRAS LPAREN expresion RPAREN NEWLINE
            // para (<tipo> <id> = <valor> ; <exp bool> ; <id> ++ --) : ...
            | PARA LPAREN tipos ID ASIG expresion PUNTO_COMA expresion PUNTO_COMA ID (INCREMENTO | DECREMENTO) RPAREN DOS_PUNTOS NEWLINE bloque
           // elegir (<exp bool>) : ...
            | ELEGITR LPAREN expresion RPAREN DOS_PUNTOS NEWLINE INDENT bloque_elegir DEDENT
            // retornar <valor>
            | RETORNAR expresion? NEWLINE
            | ROMPER NEWLINE
            | CONTINUAR NEWLINE
            ;
```

```
         // sino (<exp bool>) entonces ...                        contrario ...
bloc_si: (SINO LPAREN expresion RPAREN ENTONCES NEWLINE bloque)* (CONTRARIO NEWLINE bloque)?
         ;

bloque: INDENT instrucciones+ DEDENT
       ;

llamada: ID LPAREN (expresion (COMA expresion)*)? RPAREN
       | ID PUNTO ID LPAREN (expresion (COMA expresion)*)? RPAREN
       ;

bloque_elegir: (CASO expresion DOS_PUNTOS NEWLINE bloque)+ SIEMPRE DOS_PUNTOS NEWLINE bloque
             ;

            // <tipo> <id> = <expresion>
definiciones: (tipos | ID) ID ASIG expresion NEWLINE
           // <tipo> <id> [<valor>] = {...}
            | (tipos | ID) ID (LCORCH expresion RCORCH)+ ASIG bloc_llaves NEWLINE
           // <tipo> <id> = {...}
            | (tipos | ID) ID ASIG LLLAVE val_arreglo RLLAVE NEWLINE
           // <tipo> <id>
            | (tipos | ID) ID NEWLINE
           // <tipo> <id> [<valor>]
            | (tipos | ID) ID (LCORCH expresion RCORCH)+ NEWLINE
            ;
```

```
          // <id>.<id> = <valor>
asignaciones: ID PUNTO ID ASIG expresion NEWLINE
          // <id>.<id>[<valor>] = <valor>
           | ID PUNTO ID (LCORCH expresion RCORCH)+ ASIG expresion NEWLINE
         // <id>[<valor>] = <valor>
           | ID (LCORCH expresion RCORCH)+ ASIG expresion NEWLINE
          //<id> = <valor>
           | ID ASIG expresion NEWLINE
          // <id> ++
           | ID INCREMENTO NEWLINE
          // <id> --
           | ID DECREMENTO NEWLINE
          // <id>.<id> ++ / --
           | ID PUNTO ID (INCREMENTO | DECREMENTO) NEWLINE
          // <id>.<id>[<valor>] ++ / --
           | ID PUNTO ID (LCORCH expresion RCORCH)+ (INCREMENTO | DECREMENTO) NEWLINE
          // <id>[<valor>] ++ / --
           | ID (LCORCH expresion RCORCH)+ (INCREMENTO | DECREMENTO) NEWLINE
          // <id> += -= *= /= <valor>
           | ID (ADD_ASSIGN | SUB_ASSIGN | MUL_ASSIGN | DIV_ASSIGN) expresion NEWLINE
          // <id>.<id> += -= *= /= <valor>
           | ID PUNTO ID (ADD_ASSIGN | SUB_ASSIGN | MUL_ASSIGN | DIV_ASSIGN) expresion NEWLINE
          // <id>.<id>[<valor>] += -= *= /= <valor>
           | ID PUNTO ID (LCORCH expresion RCORCH)+ (ADD_ASSIGN | SUB_ASSIGN | MUL_ASSIGN | DIV_ASSIGN) expresion NEWLINE
          // <id>[<valor>] += -= *= /= <valor>
           | ID (LCORCH expresion RCORCH)+ (ADD_ASSIGN | SUB_ASSIGN | MUL_ASSIGN | DIV_ASSIGN) expresion NEWLINE
           ;
```

```
        // - <val>
expresion: RESTA expresion
        // ! <exp>
         | NOT expresion
        // (...)
         | LPAREN expresion RPAREN
        // <val> * / % <val>
         | expresion (MULT | DIV | MOD) expresion
         // <val> + - <val>
         | expresion (SUMA | RESTA) expresion
         // <val> == != <val>
         | expresion (IGUAL | NO_IGUAL) expresion
         // <val> <= >= <val>
         | expresion (MENOR_IGUAL | MAYOR_IGUAL) expresion
         // <val> < > <val>
         | expresion (MENOR | MAYOR) expresion
         // <val> && || <val>
         | expresion (AND | OR) expresion
        // <id>[...] <- Puede tener de 1 a n []
         | ID (LCORCH expresion RCORCH)+
        // <id>.<id>
         | ID PUNTO ID
        // <id>.<id>[...] <- campo que es arreglo
         | ID PUNTO ID (LCORCH expresion RCORCH)+
        // <id>( <valor> , <valor>)
         | ID LPAREN (expresion (COMA expresion)*)? RPAREN
        // <id>.<id>( <valor> , <valor>)
         | ID PUNTO ID LPAREN (expresion (COMA expresion)*)? RPAREN
         // leer()
         | LEER LPAREN RPAREN
         | VERDADERO
         | FALSO
         | ID
         | INT
         | DECIMAL
         | STRING
         | CHAR
         ;
```

```
tipos: CADENA
     | ENTERO
     | FLOTANTE
     | CARACTER
     | BOOL
     ;
```

Compatibilidad de tipos

Equivalencia de Tipos

Tipo Interno	Y?	PigLatin	Zetariano
ENTERO	entero	numerus	int
FLOTANTE	flotante	decimalis	double
CADENA	cadena	textum	String
CARACTER	caracter	littera	char
BOOL	bool	bool	boolean
VOID	-	-	void
SERIES	series<T>	series<T>	T[]
STRUCT	struct	structura	class
NULL	-	null	null

Asignación de Aceptación

Destino	Y?	PigLatin	Zetariano
entero / numerus /int	entero,caracter, bool	numerus	int
flotante / decimales /double	flotante,entero,caracter, bool	decimales, numerus	double, int
cadena / textum / String	cualquier primitivo: cadena, entero, flotante, caracter,	cualquier primitivo: cadena, entero, flotante, caracter,	solo String

	bool	bool	
caracter / littera /char	caracte, bool	littera	char
bool / boolean	bool	bool	boolean
struct / clase S	solo S (mismo nombre)	S o subclase de S	S o subclase de S
series<T> / T[]	series<U> : si T acepta U	series<U> : si T acepta U	u[] : si T acepta U
null	-	clase, series, textum	clase, arreglo, String

Resultados en Operaciones

Resultado	Y?	PigLatin	Zetariano
“a” + 1	cadena	textum	String
1 + 2	entero	numerus	double
1 + 1.1	flotante	decimalis	error
‘a’ + 1	error	error	error
true + 1	error	error	error
“a” - 1	error	error	error
*“a” * 1*	error	error	error
1 / 2	entero	numerus	int
1 / 1.1	flotante	decimalis	double
1 % 2	ERROR estatico	-	int
1.5 % 2	Error estatico	-	double
“a” == “n”	bool	bool	boolean
1 == 1.1	bool	bool	boolean
‘a’ == 65	bool	bool	error
true == false	bool	bool	boolean
“a” == 1	error	error	error
1 < 2.0	bool	bool	boolean

'a' < 65	bool	bool	error
"a" < "b"	error	error	error
true && false	bool	bool	boolean
1 && 2	error	error	error
- (numero)	mismo tipo	mismo tipo	mismo tipo
!b / non b	bool	bool	boolean
cond ? a : b	-	-	tipo comun o error